

树上算法

Larunatreacy

2026 年 7 月 6 日

目录

- ① 虚树
- ② 点分治
- ③ 基环树
- ④ 树上重编号
 - *bdfs* 序
 - 毛毛虫剖分
 - K -毛毛虫剖分

① 虚树

② 点分治

③ 基环树

④ 树上重编号

- $bdfs$ 序
- 毛毛虫剖分
- K -毛毛虫剖分

虚树：引入

例题：【模板】虚树 / [SDOI2011] 消耗战

给定一棵 n 个节点的树，每条边有一个切断代价 w_i 。 m 次询问，每次询问给定 k_i 个点，需要切断若干条边，使得 1 号点不能到达这些点，询问最小代价。

$$2 \leq n \leq 2.5 \times 10^5, \quad 1 \leq m \leq 5 \times 10^5, \quad \sum k_i \leq 5 \times 10^5$$

朴素做法令 dp_i 表示 i 号点不能到达子树中关键点的最小代价，复杂度为 $O(mn)$ 。

虚树：核心动机

一次 DP 过程中，大量与关键点无关的子树不会产生贡献。
由于 $\sum k_i$ 较小，我们希望每次询问只保留与关键点数量同阶的信息，把原树浓缩成一棵小树。

核心目标

保留关键点之间的祖先关系与路径信息，使得原树上的 DP 可以等价地在小树上完成。

虚树：概念

虚树中的点

- 关键点；
- 任意两个关键点的 LCA；
- 根节点 1。

虚树中的边

两个虚树点在原树上简单路径的信息压缩，例如路径最小边权、路径长度等。

若关键点个数为 k ，则虚树大小不超过 $2k - 1$ 。

虚树：二次排序 + LCA 连边

将关键点按 DFS 序排序，再把相邻点的 LCA 加入点集，去重后即可得到虚树点集。

连边时按 DFS 序扫描点集。对于相邻点 x, y ，求 $lca = \text{LCA}(x, y)$ ，则 y 在虚树上的父亲就是这个 LCA。

复杂度

排序带来单次构造复杂度 $O(k \log k)$ 。若边权信息需要倍增或树剖求出，则另计对应复杂度。

虚树：二次排序为什么正确

- DFS 序最小的点一定是根，因此没有父亲。
- 若 $LCA(x, y) = x$ ，则 x 到 y 的路径上没有其它虚树点， x 是 y 的父亲。
- 若 $LCA(x, y) \neq x$ ，则由 DFS 序性质可知 y 到 LCA 的路径上没有其它虚树点，因此该 LCA 是 y 的父亲。

每个点都找到了唯一父亲，边数为点数减一，因此不重不漏。

虚树：单调栈构造

单调栈维护虚树上的一条链。栈底到栈顶 DFS 序递增，并认为当前栈中相邻点存在临时父子关系。

按 DFS 序依次加入关键点 x ，设栈顶为 y ，求 $lca = LCA(x, y)$ ：

- 若 $lca = y$ ，说明 x 在当前链上，直接入栈。
- 否则不断弹栈并连边，直到 LCA 能被正确插入当前链。
- 只有一个点出栈时，才确定其虚树父亲并连边。

复杂度同样为 $O(k \log k)$ ，但常数通常更优。

例题：【模板】虚树 / [SDOI2011] 消耗战

每次询问建出关键点的虚树，然后在虚树上跑 DP。
虚树边权为连接两点在原树路径上的最小边代价，可用树上倍增在 $O(\log n)$ 内求出。

复杂度

$$O\left(\sum k_i \log n\right)$$

例题：「HEOI2014」大工程

题意

每次给定 k 个点，求关键点两两距离和、最小距离、最大距离。

对于求和，可以拆成边贡献：一条边被经过的次数为边两侧关键点个数乘积。

对于最小值和最大值，可在虚树上做简单 DP，在路径最上方的点拼接两条半路径。

建出虚树后一遍 DFS 即可，复杂度 $O(\sum k_i)$ 。

例题：「HNOI2014」世界树

题意

每次给定若干关键点，每个树上点归属于距离最近的关键点，若距离相同则编号最小。求每个关键点管辖的节点数量。

建出关键点虚树后，在虚树上跑多源最短路，求每个虚树点最近的关键点编号与距离。

虚树点代表若干原树点，虚树边代表路径及其附属子树。边上归属可由两端信息计算分界点，再用预处理信息容斥统计。

例题：「NOI2021」庆典

缩点后得到 DAG。利用题目特殊性质，可在不改变可达性的前提下删边，把 DAG 转化为一棵叶向树。

当临时加入边数 $k \leq 2$ 时，把 s, t 以及所有临时边端点作为关键点建立虚树。

虚树边为祖先到子孙的有向边，边权为路径中间点数；临时边作为权值为 0 的有向边加入虚树。

小图上正反各跑一次可达性，即可统计所有可能经过的边权。

例题：「SDOI2018」战略游戏

由于只删除一个点，答案只可能来自割点。

考虑原图圆方树。对于询问点集 S_i ，答案等于它们在圆方树上的最小连通子树中，不属于 S_i 的圆点个数。

可以在构造虚树时同步统计答案，复杂度为构造虚树的单 $\log$ 。

例题：「ZJOI2019」语言

问题可转化为动态维护树上若干点的最小连通块大小。

经典结论：将点按 DFS 序排序，最小连通树边权和的两倍等于相邻点距离和，其中首尾也视为相邻。

在路径 LCA 处删除两个端点，在节点处启发式合并儿子集合，并增量维护答案。

总复杂度 $O(m \log^2 m)$ 。

虚树：作业

- Bear and Chemistry
- 「AHOI / HNOI2018」 毒瘤
- 【JRKSJ R6】Dedicatus545
- 【NOI】桂花树

1 虚树

2 点分治

3 基环树

4 树上重编号

- $bdfs$ 序
- 毛毛虫剖分
- K -毛毛虫剖分

点分治：概念

点分治常用于解决树上路径或点对贡献类问题，基本复杂度为 $O(n \log n)$ 。

核心思想

对于当前树，将路径分为：

- 经过当前重心的路径；
- 不经过当前重心的路径。

先处理经过重心的路径，再删除重心，把剩余连通块递归处理。

点分治：复杂度

以重心为根时，它所有儿子的子树大小都不超过整棵树的一半。
 若处理一棵大小为 $size$ 的树复杂度为 $O(size)$ ，则每层总点数为 $O(n)$ ，递归层数为 $O(\log n)$ 。
 因此总复杂度为：

$$O(n \log n)$$

点分治：过程拆解

一般模板可拆成三步：

- 求当前连通块重心；
- 求重心各个子树大小，便于递归；
- 处理所有经过重心的路径贡献。

处理经过重心的路径时，通常把一条路径拆成两条从重心出发的半路径，并用桶、树状数组、哈希表等数据结构维护已加入的半路径信息。

点分治：处理跨重心路径

对于每个重心的儿子子树，依次执行：

- ① 递归收集当前子树中点到重心的半路径信息；
- ② 用当前子树信息与之前子树信息拼接并更新答案；
- ③ 将当前子树的半路径信息加入数据结构。

注意最后需要回滚或清空数据结构，避免影响其它分治层。

例题：[国家集训队] 聪聪可可

题意

给定带边权树，求有多少条路径满足路径边权和 $\text{mod } 3 = 0$ 。

点分治后，只需关注半路径权值模 3 的结果。

开三个桶分别记录模 0, 1, 2 的半路径数量，拼接时查询互补模数即可。

复杂度为一只 $\log$ 。

例题：Tree

题意

给定带边权树，求距离不超过 k 的点对数量。

点分治处理跨重心点对。用树状数组维护已有半路径长度数量。
当前半路径长度为 d 时，只需查询长度不超过 $k - d$ 的已加入半路径数量。

复杂度 $O(n \log^2 n)$ 。

例题：[USACO13OPEN] Yin and Yang G

把白边视为 -1 ，黑边视为 1 。如果没有分界点限制，只需统计半路径权值差的相反数。

加入分界点限制后，将已加入半路径分成两类：

- 已经存在合法分界点；
- 尚未存在合法分界点。

根到当前点路径上若存在相同前缀和，则当前半路径存在分界点。用哈希表维护即可。

例题：路径规划

题意

找一条路径，使得路径权最小值乘边权和最大。

拼接时最小值可能来自当前半路径，也可能来自已加入半路径。直接维护会较复杂。考虑固定最小值一定在当前半路径中，正反各做一次拼接过程。

用树状数组维护路径信息，复杂度 $O(n \log^2 n)$ 。

例题：[USACO18JAN] Cow at Large P

设 g_x 为 x 到最近叶子的距离。

对于起点 s ，满足 $g_x \leq \text{dis}(x, s)$ 的点形成若干极大连通子树，
每个极大子树贡献 1 个守卫。

给点 x 赋权 $2 - \deg_x$ ，则一个极大连通子树权值和为 1。

于是转化为满足 $g_u \leq \text{dis}(u, v)$ 的点对贡献问题，可用点分治解决。

例题：[WC2010] 重建计划

先二分平均值，把边权减去二分值，问题变成是否存在长度在 $[L, R]$ 中且权值和非负的路径。

点分治处理跨重心路径。相同长度只保留最大权值，按长度从小到大处理时，合法长度范围构成滑动窗口。

用单调队列优化，复杂度 $O(n \log^2 n)$ 。

点分治：作业

- **【USACO12NOV】** Balanced Trees G
- P6326 Shopping
- P9058 [Ynoi2004] rpmtdq
- **【COTS 2023】** 三元图 Graf

1 虚树

2 点分治

3 基环树

4 树上重编号

- *bdfs* 序
- 毛毛虫剖分
- K -毛毛虫剖分

基环树：概念

基环树是指有且仅有一个环的树。基环森林是若干棵基环树组成的森林。

对于有向图：

- 内向基环树：每个点有且只有一条出边；
- 外向基环树：每个点有且只有一条入边。

基环树和基环森林都有点数等于边数的特点。

基环树：核心思路

处理基环树问题的一般流程：

- ① 找出环，把环上点看作若干树根；
- ② 每个非环点只属于某个环上点的子树，先分别做树形 DP 或 DFS；
- ③ 破坏成链，把每个环上点压缩为它子树的信息，在链上 DP；
- ④ 若存在区间最优转移，可用单调队列优化。

例题：[NOIP2018 提高组] 旅行

树的情况中，答案是以 1 为根，并把每个点儿子按编号排序后的 DFS 序。

基环树中，环上至少有一条边不会被经过。

枚举断掉环上的哪条边，变成树后求对应 DFS 序，取字典序最小值。

复杂度 $O(n^2)$ 。

例题：[ZJOI2008] 骑士

题意

给定基环森林，每个点有点权，求最大点独立集。

对每棵基环树独立处理。先找环，以环上点为根分别做树形 DP。
对于环上点 x ，压缩得到：

$$f_x = \text{不选 } x \text{ 的最大值}, \quad g_x = \text{选 } x \text{ 的最大值}$$

最后在环上破坏成链，再做一次环上 DP。

例题：[IOI2008] Island

题意

给定基环森林，求每棵基环树直径之和。

路径分为两类：

- 完全在某棵环外子树中；
- 经过环边。

前者树形 DP 即可。后者对每个环上点 x 求其到子树内最远点距离 f_x ，破坏成链后用前缀和与单调队列维护最优拼接。

例题：CF1967D Long Way to be Non-decreasing

连边 $i \rightarrow b_i$ ，得到一张内向基环森林。

问题变为：每次可选择若干点沿出边移动一步，最少多少次后使序列单调不降。

二分答案 mid ，判定每个点最多走 mid 步时是否能构造非降序列。

贪心让每个 a_i 取不小于前一个值的最小可达值，并用并查集和深度信息快速判断可达性。

例题：[JOISC 2021] 最悪の記者 4

连边 $i \rightarrow a_i$ 得到内向基环森林，需要最小代价修改点权，使每个点权不小于它指向的点权。

树部分可做值域 DP，并用线段树合并优化。

对于基环树，环上点最终需要取相同值。候选值只需考虑 0 或某个环上原点权。

每棵子树合并后枚举环取值，复杂度 $O(n \log V)$ 。

基环树：作业

- 【NOIP2023】三值逻辑
- 【POI 2012】RAN-Rendezvous
- 【NOI2012】迷失游乐园
- 【JOI 2025 Final】邮局 / Post Office

① 虚树

② 点分治

③ 基环树

④ 树上重编号

- $bdfs$ 序
- 毛毛虫剖分
- K -毛毛虫剖分

树上重编号：总览

核心思想是通过重新编号，把树上的修改或查询区域变成较少的区间，从而交给数据结构维护。

本节包含：

- *bdfs* 序：支持子树操作与点邻域操作；
- 毛毛虫剖分：本质上是 1-毛毛虫剖分；
- K -毛毛虫剖分：支持子树操作与路径的 K 邻域操作。

注意

bdfs 序和毛毛虫剖分都属于树上重编号思想的一部分，不应作为与“树上重编号”并列的一级主题。

bdfs 序：适用范围

bdfs 序可以解决同时含有某个点的邻域操作和子树操作的问题。
如果存在路径操作，它通常不适用。

设最大邻域半径为 K ，单次操作复杂度一般为：

$$O(K \log n)$$

bdfs 序：重编号思想

DFS 序下，任意点子树编号连续；BFS 序下，任意子树内的 K 邻域编号连续。

bdfs 序将二者结合：

按照 DFS 顺序遍历每个点，遍历到点 x 时，把它子树内的 K 邻域按 BFS 顺序加入编号序列。

一个点只有唯一的 K 级祖先，因此每个点只会被加入一次。

bdfs 序：关键性质

设 $L_{x,i}, R_{x,i}$ 表示 x 子树内 i 邻域对应区间, st_x, et_x 表示 x 子树中 K 以内邻域以外点的区间。

- 每个点子树内任意 $k' \leq K$ 邻域都对应一个区间。
- 每个点子树中除去 K 以内邻域后的点也对应一个区间。

编号复杂度为 $O(nK)$ 。

bdfs 序：邻域操作

- 子树内的 k 邻域修改或查询：直接使用区间 $[L_{x,k}, R_{x,k}]$ 。
- k 以内邻域：拆成若干祖先方向上的子树内邻域。
- 恰好 k 邻域：若操作可容斥，则用 k 以内减去 $k-1$ 以内；否则拆成若干不交区间。

bdfs 序：子树操作

一棵以 x 为根的子树可拆成两部分：

- x 的 K 以内邻域；
- x 子树中除去 K 以内邻域后的部分。

前者做 $K + 1$ 次邻域操作，后者对应区间 $[st_x, et_x]$ 。

因此子树操作也能转化为少量区间操作。

bdfs 序：练习

- 邻域查询

毛毛虫剖分

毛毛虫剖分本质上是 1-毛毛虫剖分。

它可以支持：

- 路径操作；
- 子树操作；
- 点或路径的 1 邻域操作。

从思想上看，它是对 *bdfs* 序的扩展：通过更适合路径结构的重编号方式，把路径及其邻域拆成较少区间。

K -毛毛虫剖分：适用范围

K -毛毛虫剖分可以同时支持：

- 子树操作；
- 路径操作；
- 路径的 K 邻域操作；
- 单点邻域操作。

若最大邻域半径为 K ，单次操作复杂度通常为：

$$O(K^2 \log n)$$

或路径场景下带树剖一只 $\log$ 。

K -毛毛虫剖分：重编号思想

由于加入了路径操作，需要结合树链剖分。

按照树剖的 DFS 序遍历每条重链，把整条重链视为整体，向轻儿子子树做 *bdfs* 式重编号。

也就是说：按重链顺序，把这条重链的 K 以内邻域逐层加入编号序列。

K -毛毛虫剖分：关键性质

重编号后具有以下性质：

- 每个点除去重儿子子树后，子树内任意 $k' \leq K$ 邻域对应一个区间。
- 每条重链除去顶部 K 个点后，剩余链段的任意 $k' \leq K$ 邻域对应一个区间。
- 每个点子树除去其所在重链片段的 K 以内邻域后，剩余点对应一个区间。

重编号复杂度为 $O(nK)$ 。

K -毛毛虫剖分：基础操作

基础操作包括：

- 点 x 子树内的 k 邻域修改或查询；
- 点 x 的 k 以内邻域修改或查询；
- 重链片段的 k 以内邻域修改或查询。

这些操作最终都可拆成 $O(K^2)$ 个区间操作。

K -毛毛虫剖分：进阶操作

子树操作

利用“子树除去重链片段 K 以内邻域后连续”的性质，再补上重链片段邻域和根附近邻域。

路径 K 以内邻域操作

树剖跳重链，每次处理一个重链片段的邻域。最后单独处理 LCA 附近邻域，避免重复或遗漏。

树上重编号：练习

- QOJ7677 数据结构
- QOJ9648 数据结构
- MX MinMax